

PROYECTO DE UNA PLATAFORMA VIRTUAL

Ajoolnaj

Sistema de Gestión Institucional para un Albergue

Aplicación de Clean Code, principios SOLID y Definition of Done en un sistema modular con Laravel.

Laravel · PostgreSQL · CSS · RBAC

Integrantes del equipo · Julio 2026

Manuel David Castro Blanco
Pablo Leonel Salomon Campos
Gerardo Alexander Díaz León
Bryant Maximiliano Dzul Perez



CAMPECHE
GOBIERNO DE TODOS

SECRETARÍA DE INCLUSIÓN



JO'OLNAJ
TU TECHO

¿Qué es Ajoalnaj?

- **Problema que resuelve:** El albergue gestiona a mano ingresos y egresos (papel, sin control de camas ni alertas de permanencia).
- **Objetivo general:** Digitalizar y centralizar toda la operación en una plataforma web segura y auditable.
- **Alcance funcional:** Admisión, expedientes, estancias, camas, permanencia (alertas 3→6 meses), seguimiento clínico y egresos.
- **Control de acceso:** 7 roles (Admin, Dirección, Encargado, Trabajo Social, Enfermería, Psicología, Cocina).

Estado de Desarrollo: Backend Fases 1-7 Terminadas



Stack y arquitectura

- **Backend:** Laravel 12 · PHP 8.2 · PostgreSQL 12
- **Frontend:** Blade + Alpine.js + CSS + Vite
- **Entorno:** Laravel Sail (Docker) · Auth con Laravel Breeze
- **Seguridad:** RBAC propio (roles y permisos en BD) integrado con Gates y Políticas nativos de Laravel.
- **Arquitectura:** Modular por dominios. Cada módulo es independiente, logrando un bajo acoplamiento.



Nuestras reglas de Clean Code



REGLA 1

Nombres descriptivos en lenguaje del dominio

Por qué: El código debe leerse como el negocio del albergue.

Evita: Ambigüedad y traducción mental.

Ejemplo: RegistrarIngresoService, PersonaUsuaría.



REGLA 2

Lógica en Servicios, no en Controladores

Por qué: Controladores delgados, reglas testeables.

Evita: "Controladores Dios" y código duplicado.

Ejemplo: El servicio orquesta crear estancia y asignar cama; el controlador solo delega.



REGLA 3

Cero "Magic Numbers": constantes con nombre

Por qué: Un valor de negocio se define en un solo lugar.

Evita: Valores inconsistentes esparcidos.

Ejemplo: Estancia::MESES_PERIODO.

Nuestras reglas de Clean Code

REGLA 4 — Validación y autorización fuera del controlador

Por qué: Separar "qué datos son válidos y quién puede" de "qué hace la acción".

Evita: Reglas de negocio dispersas y fallas de seguridad.

Cómo: StoreDiagnosticoRequest define rules() y authorize().

REGLA 5 — Comentarios del "por qué" y tipos

Por qué: El código dice el "qué"; el comentario aporta intención.

Evita: Comentarios obvios que se vuelven mentiras al actualizar el código.

- **Extra:** Funciones cortas de responsabilidad única.

```
public function ejecutar(
    Expediente $expediente,
    array $datos,
    ?int $camaId = null
): Estancia {
    return DB::transaction(function () use ($expediente, $datos, $camaId) {

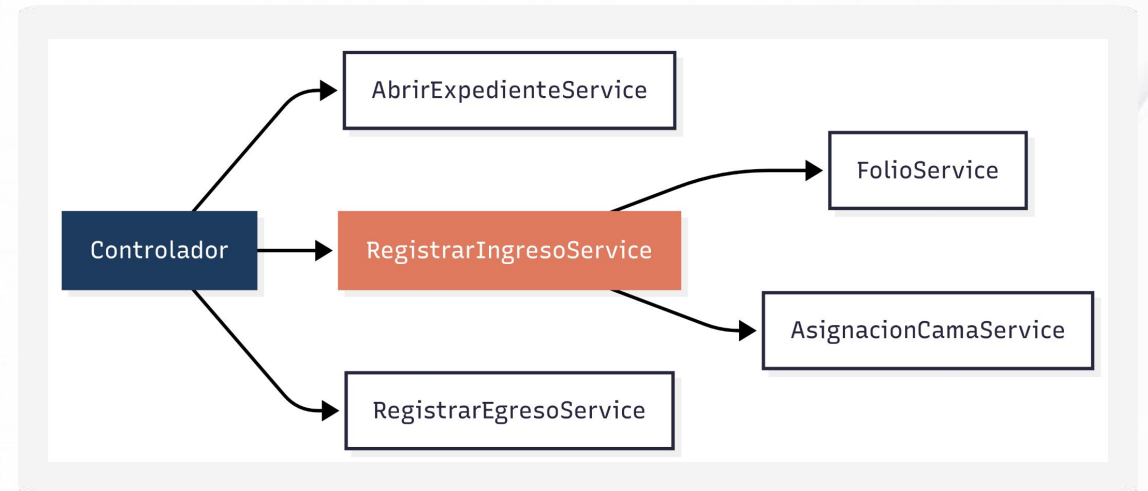
        // Reingreso = nueva estancia con episodio incrementado
        $ultimo = Estancia::where('expediente_id', $expediente->id)
            ->max('numero_episodio') ?? 0;

        $estancia = Estancia::create([
            'numero_episodio' => $ultimo + 1
        ]);

        // ... asignación de cama delegada a otro servicio
        return $estancia;
    });
}
```

SRP — Principio de Responsabilidad Única

- **En qué consiste:** Cada clase debe tener una sola razón para cambiar (una sola responsabilidad).
- **Dónde lo aplicamos:** En los Servicios de dominio. Cada servicio hace *una sola cosa*.
- **Qué evita:** Clases enormes que hacen de todo y se rompen al tocarlas.
- **Beneficio:** Podemos cambiar cómo se genera el folio sin tocar la asignación de camas, facilitando las pruebas unitarias.



DIP — Inversión de Dependencias

- **En qué consiste:** Las clases dependen de abstracciones o se les inyectan sus dependencias, no las instancian directamente.
- **Dónde lo aplicamos:** Los servicios reciben sus colaboradores por el constructor gracias al contenedor de dependencias de Laravel.
- **Qué evita:** Acoplamiento rígido (evita usar `new AsignacionCamaService()` dentro de otro servicio).
- **Beneficio:** Permite inyectar *mocks* fácilmente durante las pruebas automáticas.



Nuestra Definition of Done

Una funcionalidad está "terminada" únicamente cuando:

- ✓ 1. Cumple los requisitos funcionales.
- ✓ 2. Sigue nuestras 5 reglas de Clean Code.
- ✓ 3. La lógica de negocio está en un Service.
- ✓ 4. Tiene validación (FormRequest o Policy).
- ✓ 5. Pasa pruebas automáticas sin errores.
- ✓ 6. Fue probada manualmente en flujo real.
- ✓ 7. No presenta errores/warnings conocidos.
- ✓ 8. Pull Request revisado y aprobado por otro.
- ✓ 9. Migraciones aplican limpio y seguras.
- ✓ 10. Documentación y README actualizados.

Deuda técnica: riesgos y prevención

Riesgo Identificado	Estrategia de Mitigación
Cobertura de pruebas limitada (SQLite difiere de PostgreSQL en migraciones).	Verificar contra BD PostgreSQL real y añadir pruebas estrictas a la lógica crítica de negocio.
Fases 8 y 9 sin implementar.	Planificarlas formalmente; prohibido "parchear" los módulos actuales para forzar su funcionamiento.
Lógica en controlador (por presión de tiempo).	Respetar tajantemente el patrón <i>Service + FormRequest</i> (DoD).
Copiar-pegar entre módulos.	Reutilizar servicios y mantener el aislamiento de la arquitectura por dominios.
Estrategia global: Las fechas ajustadas causan baja calidad. Lo prevenimos con el DoD, PR obligatorios y registrando la deuda conscientemente en vez de ocultarla.	

Nuestro MVP: el ciclo de valor

MVP (Actualmente)

- Login y control de acceso por rol.
- Alta de expediente y persona usuaria.
- Seguimiento evaluaciones básico.
- Egreso de la persona usuaria.

Futuras iteraciones

- Gestión de inventarios y operación interna (Fase 8).
- Vista móvil de solo lectura para monitoreo (Fase 9).

Justificación: El MVP cubre el ciclo vital de una persona (Ingreso → Estancia → Evaluaciones → Egreso), aportando el valor central al albergue de inmediato.

Conclusión

- Ajoolnaj resuelve un flujo institucional real del albergue.
- La arquitectura por dominios reduce acoplamiento y facilita mantenimiento.
- Clean Code, SOLID y DoD funcionan como reglas de entrega, no como teoría aislada.
- El MVP entrega valor inmediato y deja una ruta clara para inventario y móvil.

